

Simulador de partículas determinístico com GPU

Ivan Ribeiro
PG55950

Francisco Ferreira
PG55942

Júlio Pinto
PG57883

Introdução

Sistemas de partículas são muito usados em computação gráfica para simular fenómenos como fumo, fogo, fluidos, etc. Estes sistemas consistem num grande número de partículas individuais, cada uma com propriedades como posição, velocidade, cor, aceleração, etc., que em conjunto produzem efeitos visuais complexos e dinâmicos. Apesar da sua versatilidade, são computacionalmente exigentes, sobretudo quando se procura maior realismo através da simulação de interações físicas e colisões entre partículas.

Neste projeto, optamos por implementar um sistema de partículas com uma abordagem baseada em física mas não necessariamente realista, incluindo deteção e resposta a colisões. Desenvolvemos o código usando Rust e WGPU. Isto permitiu explorar APIs e linguagens modernas, enquanto tendo controlo total sobre a o pipeline de renderização e execução.

Algoritmos de integração

Para poder simular os movimentos das partículas, precisamos de integrar o mesmo, ou seja, usar passos discretos para simular um movimento que na vida real seria contínuo, baseando-nos nas equações de movimento.

O grupo investigou algumas alternativas, tais como:

- Basic Verlet Integration
- Velocity Verlet
- Euler integration
- Leapfrog Integration
- Position Verlet
- Runge-Kutta Methods (e.g., RK4)
- Backward Euler

Tínhamos como prioridade obter boa performance mas também estabilidade razoável da simulação, pelo que optamos por usar Basic Verlet, que nos pareceu um bom compromisso.

Assim, cada partícula tem de conter estes dados:

```
struct ParticlePhysics {  
    pos: Vec2,  
    old_pos: Vec2,  
    accel: Vec2,  
}
```

E, em cada frame, a sua posição será atualizada com

```
let vel = particle.pos - particle.old_pos;  
particle.old_pos = particle.pos;  
let accel = particle.accel;  
particle.pos += vel + (accel * DELTA_SQUARED); // DELTA_SQUARED == dt * dt  
particle.accel = Vec2::ZERO;
```

Colisões

Algoritmo de colisão

Para determinar as colisões entre partículas, comparamos as posições de todas as partículas com as de todas as outras partículas. Para calcular a possível colisão entre duas partículas, `particle_a` e `particle_b`, são efetuados os seguintes cálculos:

```
const MIN_DIST: f32 = PARTICLE_DIAM;  
const MIN_DIST_SQUARED: f32 = MIN_DIST * MIN_DIST;  
const AVOID_NAN: f32 = 0.0001;  
  
let axis: vec2f = particle_a.pos - particle_b.pos; // AB ou BA vai dar ao mesmo  
let dist_squared: f32 = dot(axis, axis);  
  
if (dist_squared < MIN_DIST_SQUARED && dist_squared > AVOID_NAN) {  
    let dist: f32 = sqrt(dist_squared);  
    collision_axis = collision_axis / dist;  
  
    let delta: f32 = 0.5 * RESPONSE_COEF * (dist - MIN_DIST);  
  
    (*particle_a).pos -= collision_axis * (0.5 * delta);  
    (*particle_b).pos += collision_axis * (0.5 * delta);  
}
```

Este algoritmo baseia-se em detetar se duas partículas se estão a intersestar uma à outra, e aplicar uma aceleração de modo a que estas se afastem, consoante o eixo de colisão.

Determinismo

Ao usar um Δt fixo, a simulação usando Basic Verlet passa a ser determinística. Esta propriedade pareceu-nos extremamente interessante, visto que o mesmo conjunto de dados de input irá sempre gerar o mesmo output. Assim, por exemplo, poderíamos gerar imagens dando cores às partículas: como elas iriam sempre calhar nos mesmos sítios, as cores iniciais poderiam ser previstas de modo a representarem as cores das imagens.

Atingir uma simulação eficiente mas também determinística tornou-se, assim, o objetivo deste trabalho.

Otimizações

O algoritmo atual percorre N^2 partículas para detetar colisões, comparando todas as partículas com todas as outras partículas. Isto é muito ineficiente, e poderá ser melhorado tirando proveito do facto de que colisões entre partículas distantes não têm de ser computadas pois não irão surtir qualquer efeito. Para além disso, não é paralelizável, deixando de lado possíveis ganhos de performance.

Atualmente, uma simulação com 10k partículas atinge 4FPS.

Binning

Para aproveitar o facto de colisões só surtirem efeito se ocorrerem entre partículas próximas, podemos usar um algoritmo de binning, dividindo o espaço numa grelha (em bins) e colidindo apenas partículas entre células adjacentes.

Assim, no início de cada passo de simulação, o algoritmo coloca cada partícula numa célula da grelha com base na sua posição. Para calcular colisões, basta, para cada célula, iterar as suas partículas, calculando colisões apenas com as partículas dentro da própria célula e nas células que a rodeiam.

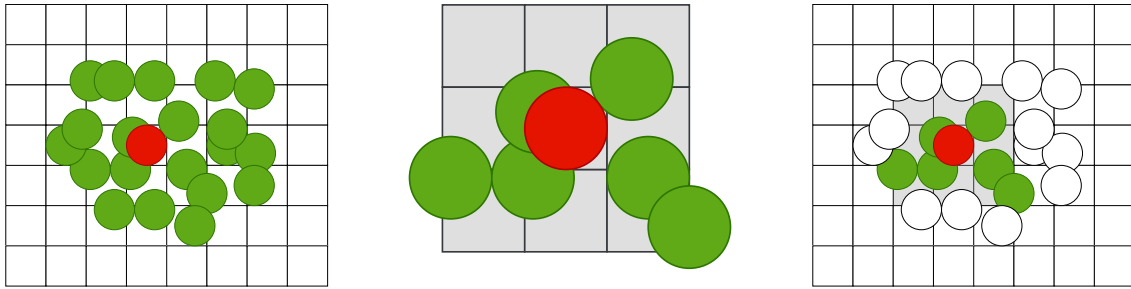


Figure 1: Partículas com que a partícula vermelha precisa de ser comparada.

Esquerda: sem binning. Meio: partículas em células adjacentes. Direita: com binning.

Com esta otimização, foi possível atingir 300FPS.

Multithreading

Visto o espaço agora estar organizado numa grelha, é possível implementar multithreading neste algoritmo, atribuindo uma parte da grelha a cada thread.

No entanto, diferentes threads podem aceder a células adjacentes, provocando data races nas colisões entre as partículas das mesmas. Estas data races tornam o algoritmo não determinístico, o que vai contra o nosso objetivo.

Para voltar a tornar a simulação determinística, decidimos fazer 2 passes de simulação distintos, procurando evitar que threads diferentes possam estar a processar a mesma célula:

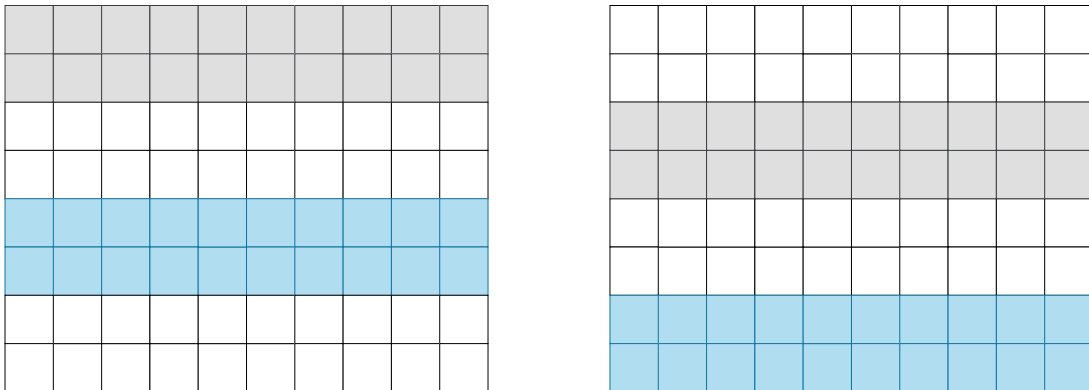


Figure 2: Os espaços cinzento e azul estão a ser processados por duas threads distintas. São feitos dois passes (esquerda e direita), garantindo que ao processar células azuis e cinzentas nunca há células adjacentes a serem processadas em simultâneo

Usando multithreading, pudemos aumentar a escala da simulação para 50k partículas, atingindo 50FPS na mesma.

Padding

Ao calcular quais as células que estão em redor de uma dada célula, existem várias posições na grelha que precisam de cuidados especiais, o que complica o código, como as células que se situam nas edges da grelha, e especialmente nos cantos. Seria necessário ter casos especiais em que determinávamos se se trata de uma destas situações, e impedir, por exemplo, a comparação com células acima devido a estas não existirem.

Assim, decidimos acrescentar uma camada de padding, ou seja, bins que nunca irão conter qualquer partícula nem ser processadas, de forma a uniformizar o código e remover condições desnecessárias do mesmo.

Esta otimização torna-se especialmente relevante para evitar thread divergence, que será útil para poder executar a detecção de colisões na GPU.

Compute shaders

Concluimos que com a capacidade computacional de uma GPU seria possível aumentar significativamente o número de partículas da simulação.

Colisão básica

Numa primeira fase, decidimos implementar novamente o algoritmo ineficiente que compara todas as partículas com todas as outras partículas.

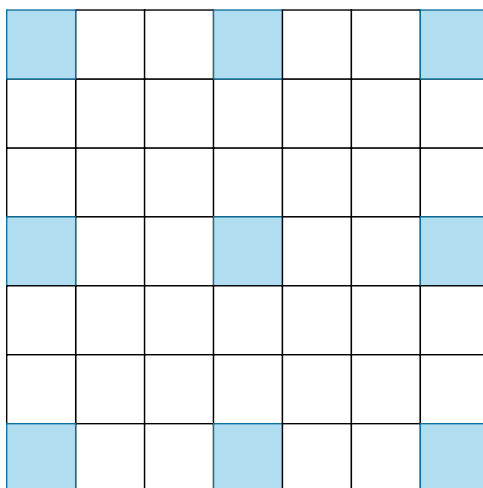
Mesmo com este uso básico da GPU, a simulação com 50K partículas atinge 30FPS.

Binning

A solução ideal, usando binning, traz uma nova dimensão aos problemas encontrados na implementação de multithreading. Tendo uma arquitetura paralela, evitar race conditions torna-se desafiante, mas necessário.

Infelizmente, para manter o determinismo, o passo de gerar as bins terá sempre de ser feito no cpu, ou pelo menos sorted no mesmo, para garantir que as partículas são sempre processadas na mesma ordem, pelo que não conseguimos evitar perdas vindas da latência de transferência de dados GPU↔CPU.

Nas colisões, a solução de usar vários passes diferentes, na solução de multithreading, terá de ser adaptada à arquitetura da GPU: não é razoável que uma thread processe uma zona inteira do espaço da grelha, mas sim uma única célula. Assim, para cada célula, teremos de garantir que existe um espaço de 2 células livre tanto na vertical como na horizontal:



Enquanto na solução com multithreading seriam suficientes 2 passes de simulação, aqui serão necessários 9 para que todas as células sejam devidamente processadas:

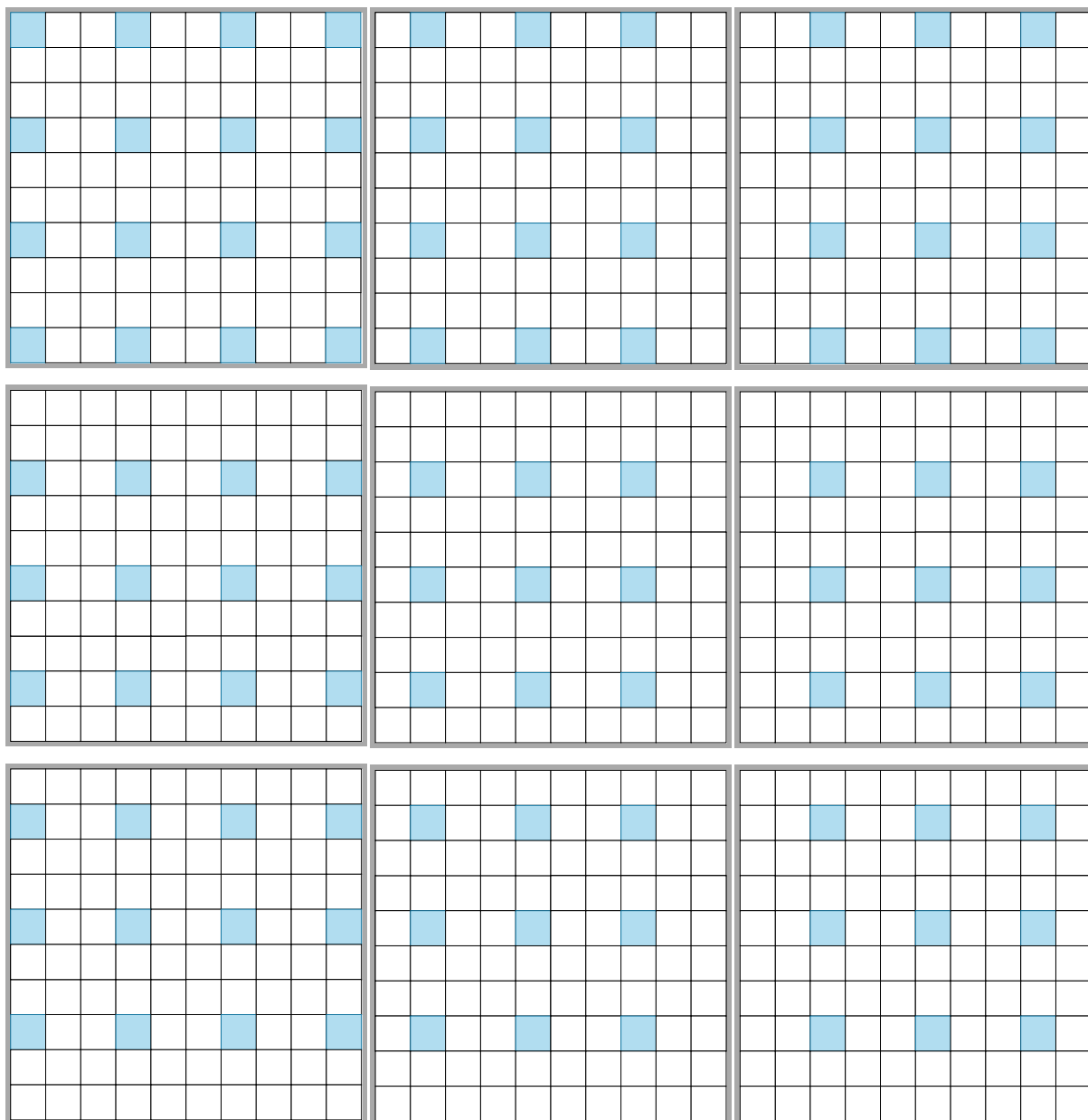


Figure 4: Os 9 passes necessários para calcular as colisões de todas as células, evitando data races

Com esta solução mais otimizada, foi possível voltar a aumentar a escala, para 100k partículas, enquanto atingimos 120FPS.

Renderização

Cada partícula é composta por 2 triângulos, formando uma quad. Visto todas serem exatamente iguais, apenas instanciamos esta quad para desenhar todas as partículas.

No vertex shader, estas são redimensionadas, e colocadas na posição correta do ecrã com base na sua posição da simulação. Esta posição já está presente na GPU através de um SSBO partilhado com o compute shader, evitando assim a latência de partilhar dados entre GPU e CPU.

No fragment shader, apenas aplicamos a textura de um círculo para que a partícula não seja renderizada como um quadrado, aplicando também uma cor.

mostrar contas de como meter na posição correta no ecrã?? e explicar como usei pixéis para fazer as coisas

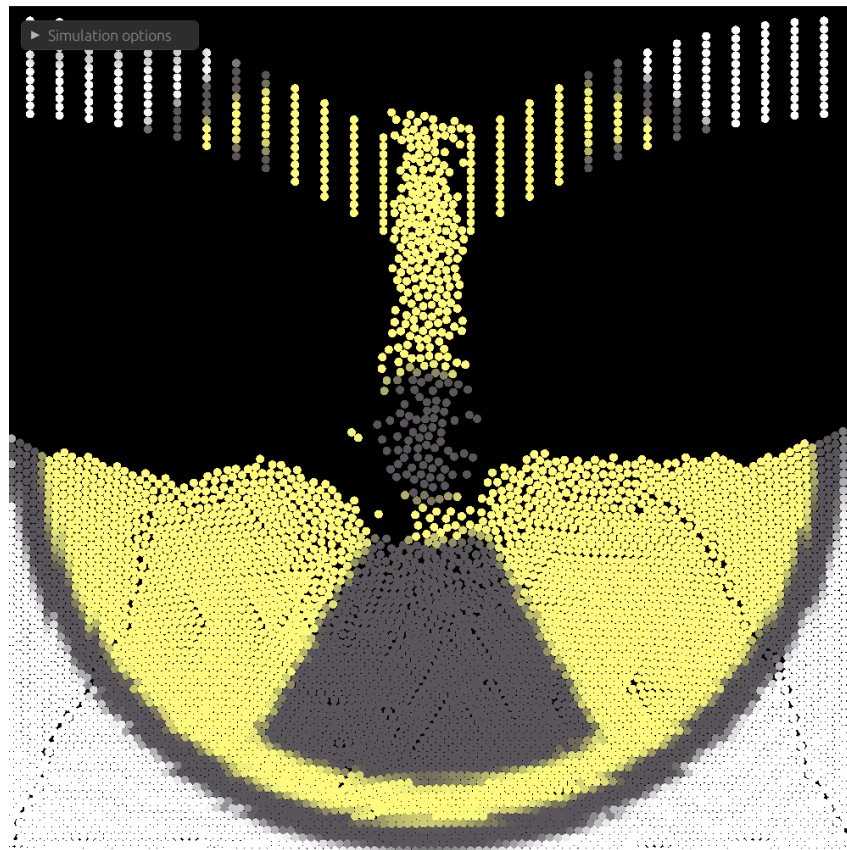
Carregar imagens

Para usarmos a nossa simulação para criar imagens com as partículas, será necessário poder atribuir às mesmas uma cor com base na sua posição.

Assim, quando se pretende carregar uma imagem, fazemos os seguintes passos:

- Decodificar imagem para RGB32
- Redimensionar imagem para a resolução da grelha de partículas
- Para cada partícula, calcular a sua posição normalizada (0-1) na grelha, e atribuir a cor correspondente da imagem

Com isto, foi possível atribuir as cores da imagem às partículas:



Spawners

Para ter maior controlo sobre a criação de partículas, decidimos desenvolver spawners:

```
struct Spawner {  
    start_frame: u64,  
    end_frame: u64,  
    spawn_every_n: u64,  
  
    spawner_type: SpawnerType, // direction, position, ...  
}
```

Estes permitem escolher em que frames começamos e paramos de criar partículas, de quantos em quantos frames criamos uma partícula, definir uma posição e direção iniciais, bem como usar um SpawnerType para codificar diferentes tipos de spawners. Por exemplo, alguns poderão ser simples e criar partículas num ponto estático, e outros poderão gerá-las num círculo em redor da simulação.

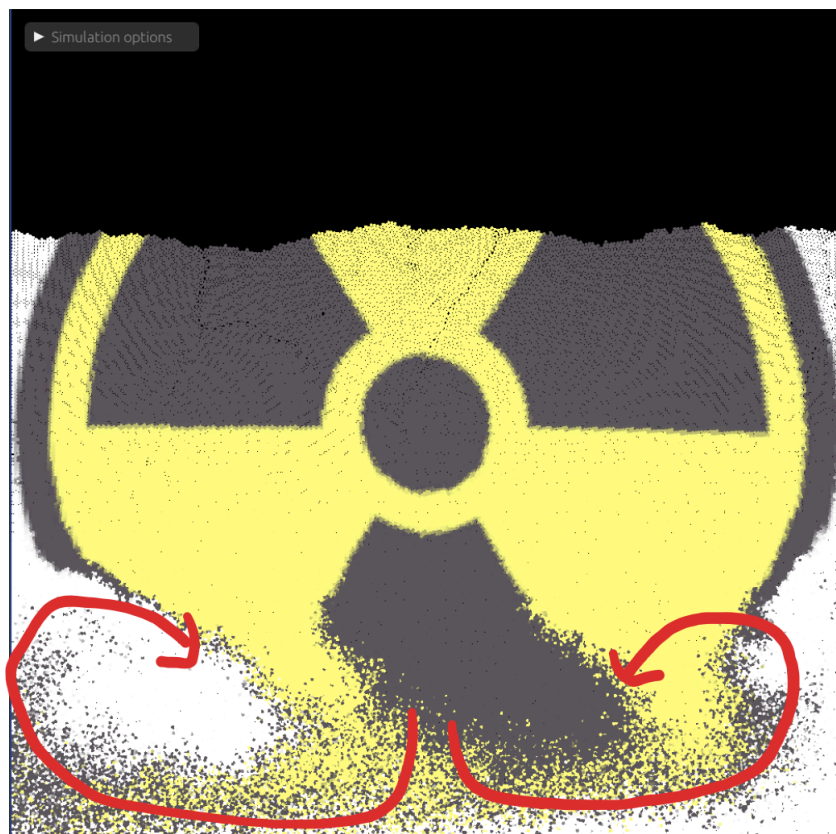
Com isto, torna-se possível customizar várias sequências de criação de partículas.

Resultados e estabilidade

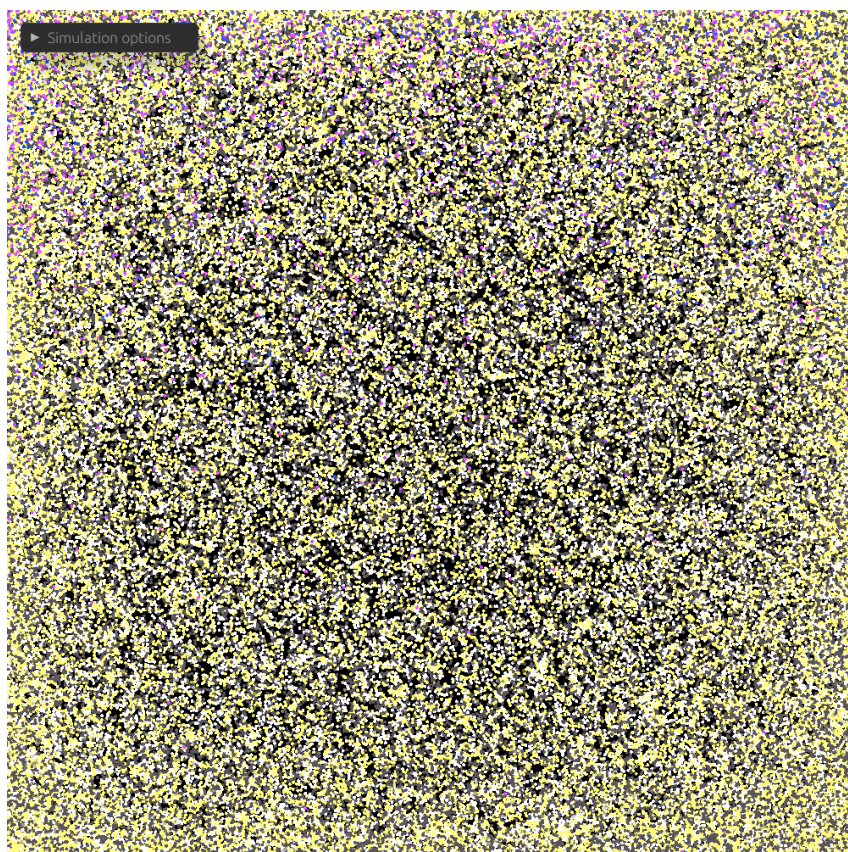
Com a ajuda do processamento na GPU, foi possível usar uma grelha 500*500, com de 200000 partículas:



No entanto, ao aumentar o número de partículas para esta escala, as partículas no fundo ficavam sob bastante “pressão”. A gravidade, ao agir sobre todas as partículas em cima das mesmas, cria colisões que constantemente empurram as partículas cada vez mais agressivamente para baixo, criando um efeito semelhante a correntes de convecção:



Para além disto, surgia também um ponto crítico, em que uma partícula sob pressão suficiente seria impulsionada com grande velocidade, atingindo outra partícula que também seria impulsionada, ..., criando um efeito de explosão em cadeia:



Apesar de efeitos interessantes, queríamos obter uma simulação que chegasse a um estado de descanso, estável, para que a imagem final fosse perceptível.

Assim, decidimos introduzir fricção na Verlet Integration. No cálculo da velocidade, introduzimos na mesma um coeficiente de 0 a 1, diminuindo artificialmente a velocidade da partícula, mesmo que esta não esteja em contacto com outras partículas.

De seguida, reduzimos também a própria gravidade, aliviando o problema da pressão. Apesar de não ser realista, a falta de perceção de escala faz com que não seja perceptível qual o efeito correto ou esperado da gravidade, fazendo com que a simulação continue agradável mesmo com gravidade muito mais fraca.

Por fim, decidimos introduzir substeps para tornar as próprias colisões mais estáveis. Em vez de simular uma vez por frame, com um dado Δt , simulamos N vezes em cada frame, usando

$$\frac{\Delta t}{\text{substeps}}$$

como o novo tempo de simulação, permitindo efetuar as computações em passos mais pequenos, tornando mais improvável que partículas, por se deslocarem demasiado rápido, passem uma por dentro da outra ou penetrem demasiado uma na outra antes que seja detetada uma colisão, gerando uma resposta violenta.

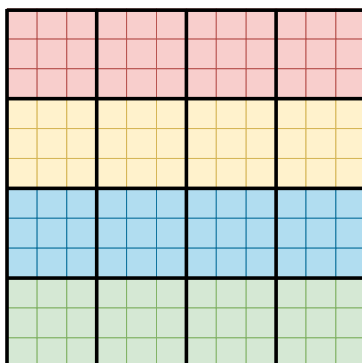
Com estas técnicas, atingimos os nossos objetivos, tendo uma simulação com uma escala satisfatória, determinística e estável, como pode ser visto em https://youtu.be/3D_PHN3UrIs

Trabalho futuro

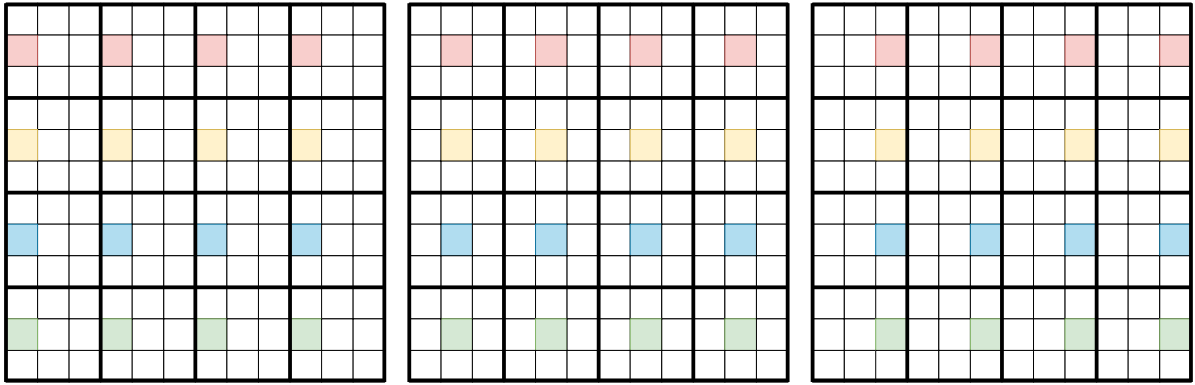
Melhorar computação de colisões na GPU

A técnica mencionada de usar 9 passes de simulação tem algumas ineficiências. Para além da elevada latência na necessidade de sincronizar a GPU 9 vezes, não consideramos a localidade entre as várias threads, abstraindo o conceito de workgroups. Para corrigir isto, concebemos um algoritmo alternativo, que infelizmente não tivemos tempo de implementar, que tira proveito da sincronização ao nível do workgroup:

Cada workgroup fica responsável por uma secção da grelha, e as suas threads processam uma secção 3x3. Neste exemplo, 4 workgroups de cores diferentes possuem 4 threads cada um:



De seguida, fazemos 3 passes, tendo cada um deles 3 pontos de sincronização ao nível do workgroup. Por exemplo, o primeiro passe poderia ser algo como:



Em que entre cada uma das etapas se usa sincronização ao nível do workgroup.

Acreditamos que este algoritmo possa mostrar melhorias de performance, enquanto mantém o determinismo ao evitar race conditions, mas infelizmente não foi implementado.

Melhorar usabilidade dos spawners

Pretendemos também, no futuro, usar a modularidade dos spawners para permitir que estes sejam colocados na simulação dinamicamente, através de uma UI.

Contar o número de partículas na GPU

Embora o binning não possa ser feito na GPU, o passo de contar o número de partículas por bin poderá ser feito através de um algoritmo de reduce paralelo.